ELO320 - Estructuras de Datos y Algoritmos
Introducción a C
Punteros

Dr. Nicolás Gálvez R.
nicolas.galvez@usm.cl

Ing. Civil Telemática - Departamento de Electrónica

1er Semestre 2025

UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

Clase Activa: Módulo 1

Ejercicio 1.1

Cree una función que reciba como parámetro un arreglo de enteros y el tamaño de éste, y luego muestre por pantalla los valores almacenados en dicho arreglo. Puede usar una de las dos siguientes declaraciones:

```
void imprimir_datos(int arreglo[], int tam);

void imprimir_datos(int * arreglo, int tam);
```

Aprendemos: Definir un arreglo, recorrer un arreglo, rellenar un arreglo. Paso por referencia.

Clase Activa: Módulo 1

Ejercicio 1.2

Cree una función que reciba como parámetro un arreglo de enteros y el tamaño de éste, y luego muestre por pantalla los valores almacenados en dicho arreglo. **No puede usar el operador []**. Puede usar una de las dos siguientes declaraciones:

```
void imprimir_datos_2(int arreglo[], int tam);

void imprimir_datos_2(int * arreglo, int tam);
```

Aprendemos: Aritmética de Punteros.

Clase Activa: Módulo 2

Ejercicio 2.1

Cree una función que reciba como parámetro un arreglo de enteros y el tamaño de éste, y luego calcule el promedio entre todos los valores almacenados en dicho arreglo. Puede usar una de las dos siguientes declaraciones:

```
float promedio(int arreglo[], int tam);

float promedio(int * arreglo, int tam);
```

Aprendemos: Operar sobre un arreglo, recorrer un arreglo.

Clase Activa: Módulo 2

Ejercicio 2.2

Cree una función que reciba como parámetro un arreglo de enteros, el tamaño de éste, y una variable que contendrá el promedio de éste arreglo. La función debe calcular el promedio entre todos los valores almacenados en dicho arreglo. El promedio debe permanecer fuera de la función. **No puede usar variables globales**. Puede usar una de las dos siguientes declaraciones:

```
void promedio(int arreglo[], int tam, float * prom);

void promedio(int * arreglo, int tam, float * prom);
```

Aprendemos: Operar sobre un arreglo, Paso de parámetro por referencia.

Clase Activa: Módulo 3

Analice y explique qué realiza el siguiente código:

Ejercicio 3.1

```
#include<stdio.h>

void swap(int a, int b){
        int temp;
        temp = a;
        a = b;
        b = temp;
}

int main(){
        int x=2, y=3;
        swap(x,y);
        printf("x: %d, y: %d\n", x, y); // ¿Qué imprime?
        return 0;
}
```

Aprendemos: Paso de parámetro por valor.

Clase Activa: Módulo 3

Analice y explique qué realiza el siguiente código:

Ejercicio 3.2

```
#include<stdio.h>

void swap(int *a, int *b){
        int temp;
        temp = *a;
        *a = *b
        *b = temp;
}

int main(){
        int x=2, y=3;
        swap(&x,&y);
        printf("x: %d, y: %d\n", x, y); //¿Qué imprime?
        return 0;
}
```

Aprendemos: Paso de parámetro por valor.

Clase Activa: Módulo 3

Analice y explique qué realiza el siguiente código:

Ejercicio 3.3

```
#include<stdio.h>

void swap(int *a, int *b){
        int *temp;
        temp = a;
        a = b;
        b = temp;
}

int main(){
        int x=2, y=3;
        swap(&x,&y);
        printf("x: %d, y: %d\n", x, y); //¿Qué imprime?
        return 0;
}
```

Aprendemos: Paso de parámetro por valor.

Clase Activa: Módulo 3

Analice y explique qué realiza el siguiente código:

Ejercicio 3.4

```
#include<stdio.h>

void swap(int *a, int *b){
        int *temp;
        temp = a;
        *a = *b;
        *b = *temp;
}

int main(){
        int x=2, y=3;
        swap(&x,&y);
        printf("x: %d, y: %d\n", x, y); //¿Qué imprime?
        return 0;
}
```

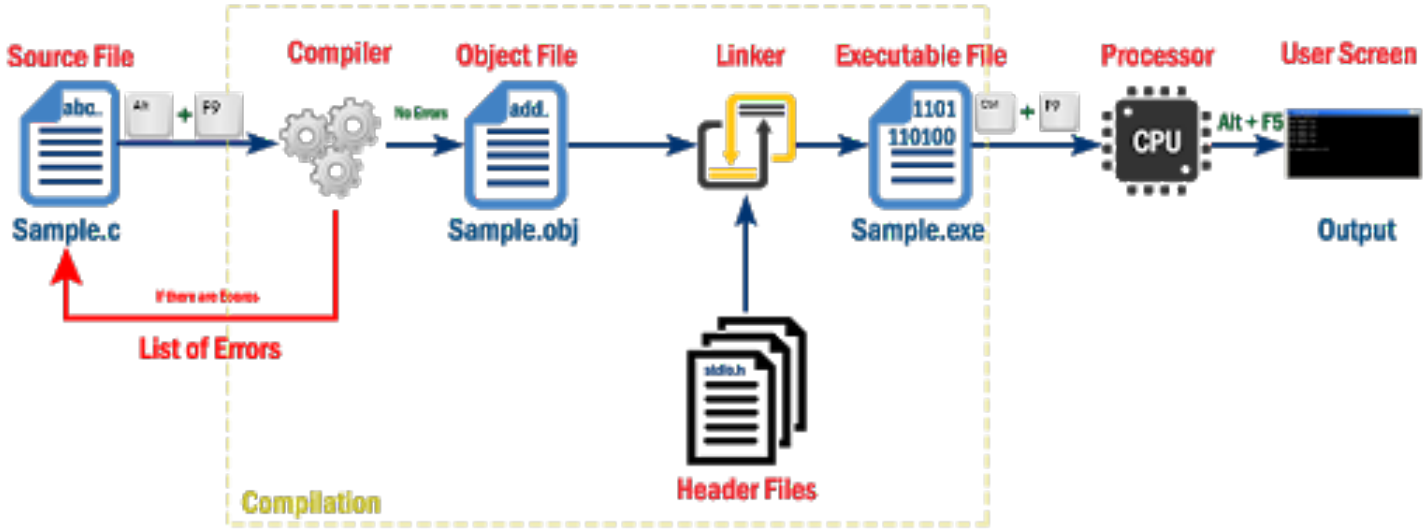
Aprendemos: Paso de parámetro por valor.

Información de Vital Importancia para la Clase Activa[1].

[1] Depende exclusivamente de uds. leerla y comprenderla.

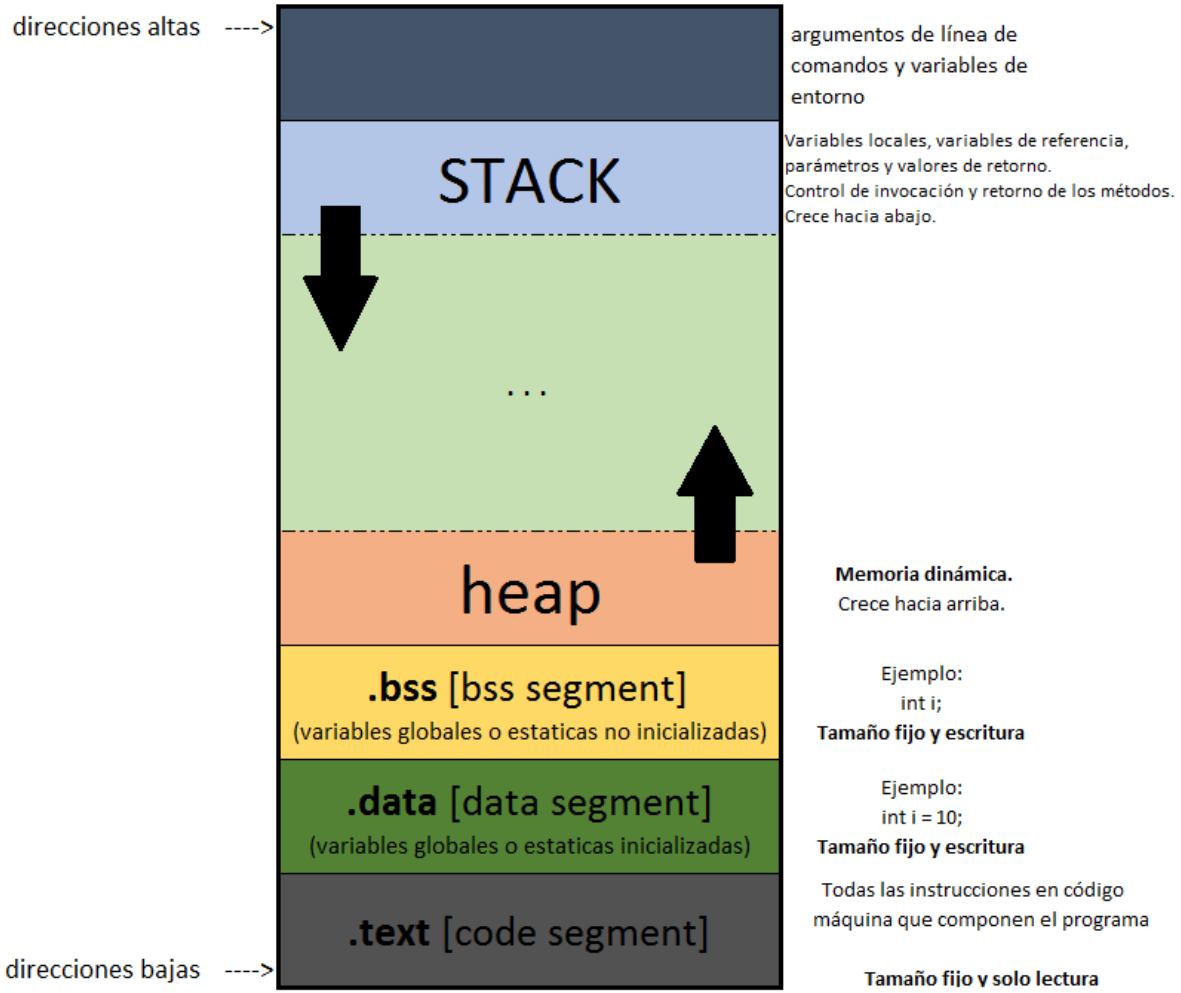
Programa en Memoria

- *Ejecutar*: llevar un programa (binario) a memoria.
- *Variables y Constantes*: espacios de que se asignan y liberan en memoria
- *Ciclo de vida*: cuando se crea y destruye una variable
- *Ámbito*: qué código tiene acceso a la variable



Taxonomía de la memoria

Segmentos de memoria:



Taxonomía de la memoria

Segmentos de memoria:

1. **Memoria text o de código**.
2. **Memoria static o estática**.
3. **Memoria heap**.
4. **Memoria stack o call stack**.

Memoria text

Donde el programa compilado se mantiene en memoria.

Memoria static

Mantiene variables globales y estáticas.

Ligado estático: variables (y valores) en la memoria hasta que programa termina.

Memoria heap

Permite al programador crear y eliminar objetos de memoria dinámicamente.

Son referenciadas indirectamente mediante **punteros** y/o **referencias**.

Memoria stack

Área dinámica de memoria donde se guardan las variables locales y la información de las funciones.

Permite mantener objetos que se asignan y liberan automáticamente en ambiente de ejecución.

Manejado **automáticamente** por el compilador.

Variables estáticas

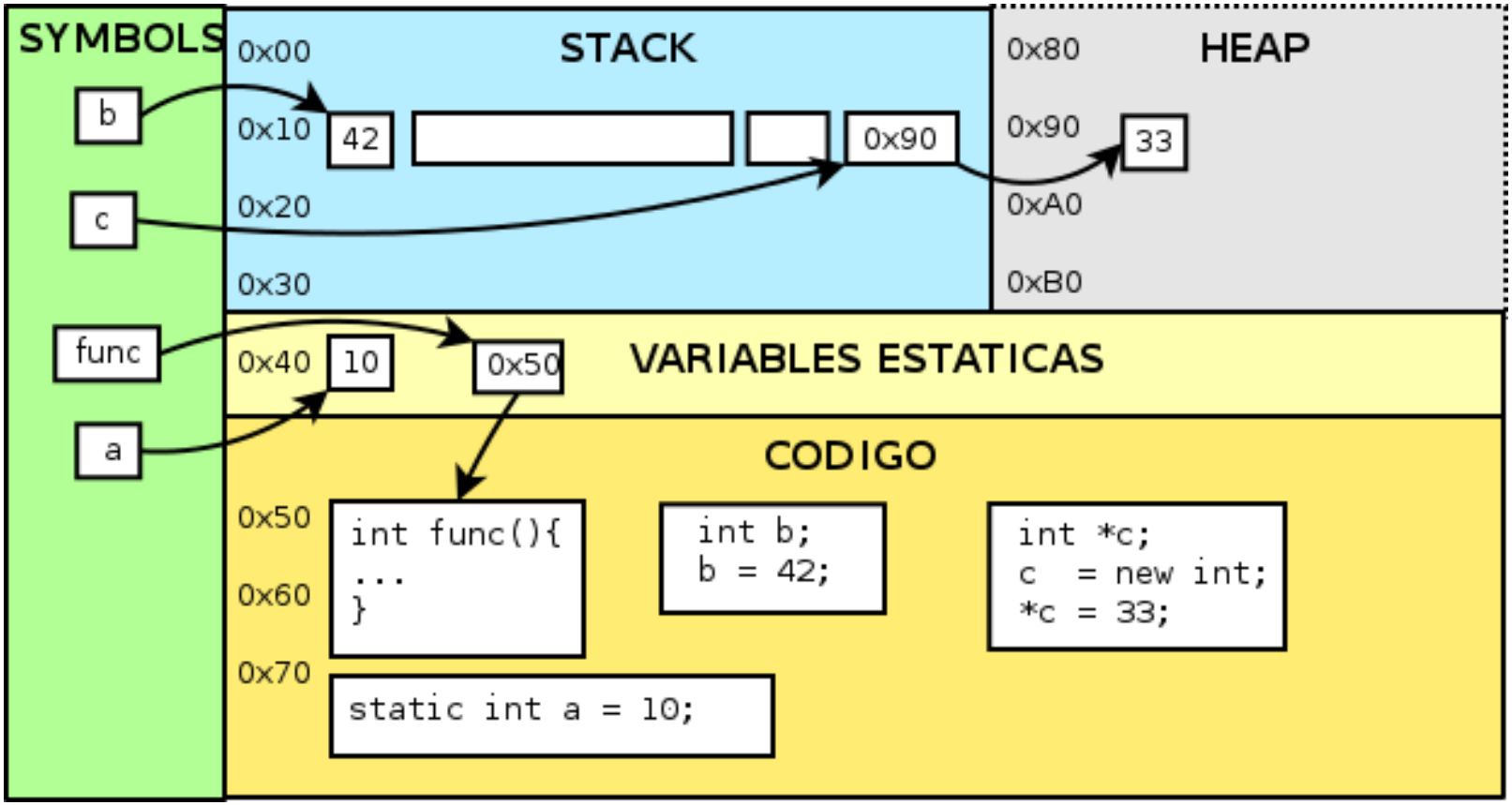
- Ligado a la memoria estática.
- Permanece durante todo el tiempo de ejecución del programa.
- **Ventajas**
  - Útil para definir variables globales y compartidas.
  - Para variables sensibles a la historia de un subprograma (e.g. modificador *static*).
  - Acceso directo permite mayor eficiencia.
- **Desventajas**
  - Impide compartir memoria entre diferentes variables (no se reasigna).

Variables estáticas - static

C

```
long y; //var global.

static int z = 1; //global estatica;
                  //solo visible a este archivo.

int est3_st4tic(){
        static int a=0; //local estatica
        a++; // no pierde su valor al finalizar ámbito
        return a;

}

int fun() {
        int i;
        for(i=0;i<10;i++){
                printf("vuelta numero %d\n", i);
        //      printf("a es imposible de imprimir: %d\n", a);
                printf("pero no pierde su valor: %d\n", est3_st4tic());
        }

        return 0;
}
```

Ejemplo


SYMBOLS
b
c
func
a
STACK
0x00
0x10
42
0x90
0x20
0x30
HEAP
0x80
0x90
33
0xA0
0xB0
VARIABLES ESTATICAS
0x40
10
0x50
CODIGO
0x50
0x60
0x70
int func(){
...
}
int b;
b = 42;
int *c;
c = new int;
*c = 33;
static int a = 10;

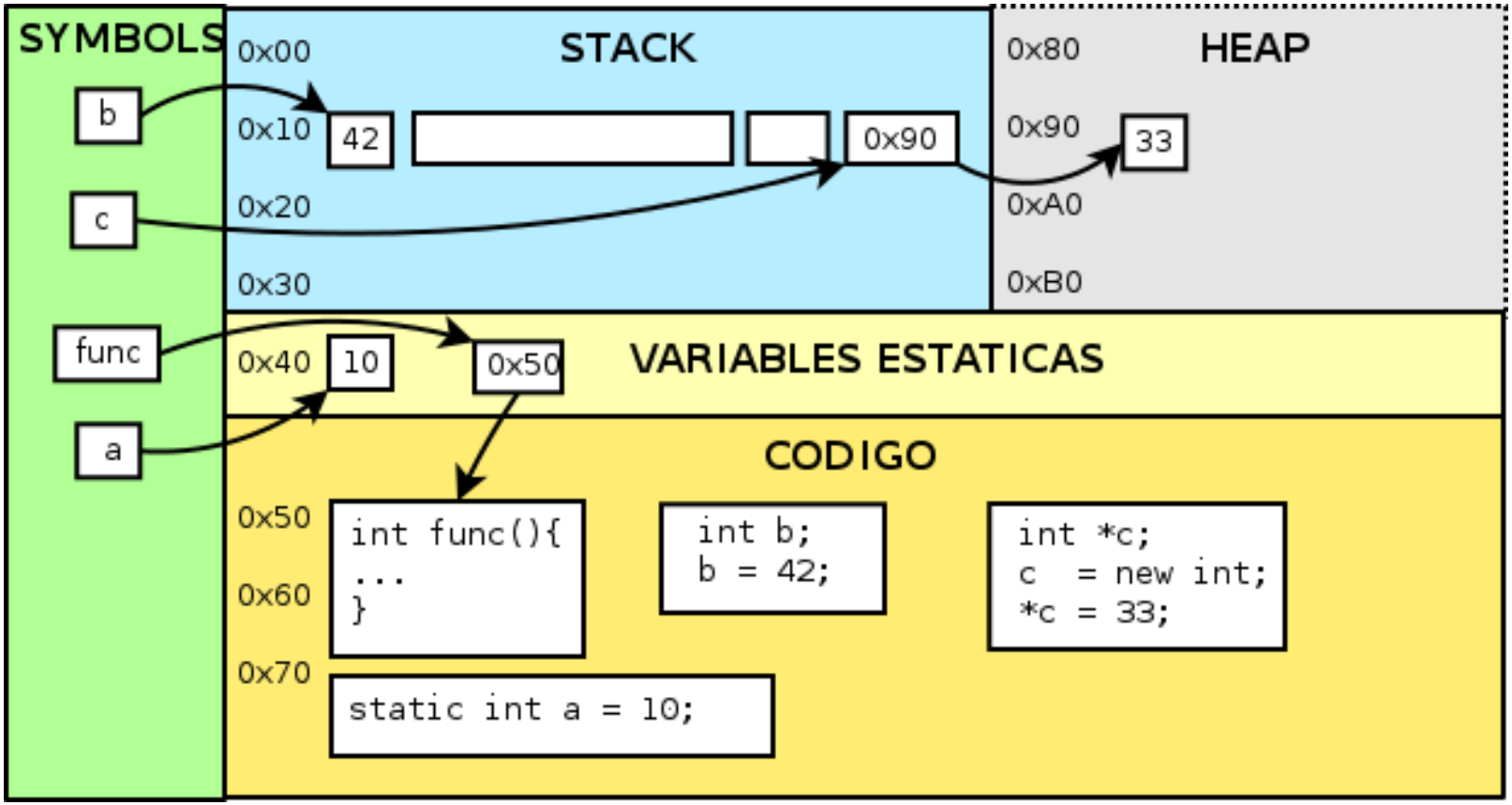
Variables dinámicas de stack

- Se asignan dinámicamente a memoria cuando se ejecuta el ámbito y la declaración asociada.
- Se liberan cuando termina su ámbito.
- Soportan bien la recursión.
- **Ámbitos:** procedimento, función, método, bloque.
- En C, variables definidas en una función (variables automáticas).

C

```
int fun() {
    int a; // Acá se crea
    {
        float b; //Acá se crea
    }//Acá se destruye b
} // Acá se destruye a
```

Ejemplo


SYMBOLS
b
c
func
a
STACK
0x00
0x10
42
0x90
0x20
0x30
HEAP
0x80
0x90
33
0xA0
0xB0
VARIABLES ESTATICAS
0x40
10
0x50
CODIGO
0x50
0x60
0x70
int func(){
...
}
int b;
b = 42;
int *c;
c = new int;
*c = 33;
static int a = 10;

Punteros en C

- Variables de stack, que guardan **direcciones de memoria**.
- Peso uniforme en sistemas modernos: 4 bytes in 32-bits, 8 bytes in 64-bits.
- Gestión de memoria dinámica (heap) y direccionamiento (stack/static)
- Pero, **¿qué es una dirección de memoria?**

Dirección de Memoria o Referencia

- Las variables de stack son cajones que pueden guardar valores, pero ¿dónde están esos cajones?.

C

```c
#include<stdio.h>

int main(){
        int b;
        scanf("%d", &b); //¿Por qué &b?

        return 0;
}
```

- El operador & (dirección de, o referencia) nos entrega la ubicación de memoria donde se encuentra el cajón.

Analogía - Biblioteca

- Va a devolver el libro **¿Cómo pasar EDA sin morir en el intento?**.
- Éste va en el casillero **Problema de Universitarios**.

Analogía - Biblioteca

- Valor 320 como identificador del libro.
- pu → Casillero.
- &pu → Ubicación Casillero → Pasillo 3, Estante 2, Fila 4.

Analogía - Biblioteca

- Valor 320 como identificador del libro.
- pu → Casillero.
- &pu → Ubicación Casillero → Pasillo 3, Estante 2, Fila 4.

Analogía - Biblioteca

- Valor 320 como identificador del libro.
- pu → Casillero.
- &pu → Ubicación Casillero → Pasillo 3, Estante 2, Fila 4.

Analogía - Biblioteca

- Valor 320 como identificador del libro.
- pu → Casillero.
- &pu → Ubicación Casillero → Pasillo 3, Estante 2, Fila 4.

Analogía - Biblioteca

- Valor 320 como identificador del libro.
- pu → Casillero.
- &pu → Ubicación Casillero → Pasillo 3, Estante 2, Fila 4.

Analogía - Biblioteca

- Valor 320 como identificador del libro.
- pu → Casillero.
- &pu → Ubicación Casillero → Pasillo 3, Estante 2, Fila 4.

Analogía - Biblioteca

- Valor 320 como identificador del libro.
- pu → Casillero.
- &pu → Ubicación Casillero → Pasillo 3, Estante 2, Fila 4.

Analogía - Biblioteca

- Valor 320 como identificador del libro.
- pu → Casillero.
- &pu → Ubicación Casillero → Pasillo 3, Estante 2, Fila 4.

Analogía - Biblioteca

- Valor 320 como identificador del libro.
- pu → Casillero.
- &pu → Ubicación Casillero → Pasillo 3, Estante 2, Fila 4.

Analogía - Biblioteca

- `pu = 320;` Bibliotecario (programador) sabé donde está el casillero.
- `scanf("%d", &pu);` Lector no sabe donde está el casillero → el bibliotecario le indica.

Analogía - Biblioteca

- `pu = 320;` Bibliotecario (programador) sabé donde está el casillero.
- `scanf("%d", &pu);` Lector no sabe donde está el casillero → el bibliotecario le indica.

Analogía - Biblioteca

- `pu = 320;` Bibliotecario (programador) sabé donde está el casillero.
- `scanf("%d", &pu);` Lector no sabe donde está el casillero $\rightarrow$ el bibliotecario le indica.

Punteros en C

- Se definen con el modificador *, e indicando su tipo.

C

```
int utfsm1 = 1680; //Dos variables de tipo int.
int utfsm2 = 3939;

int *dir = NULL; //Puntero a un int, parte en NULL
//un puntero guarda la dirección de memoria de una
//variable o segmento de memoria de tipo int.
```

- Diremos que un puntero **apunta** a la dirección de memoria que almacena.

Valor NULL

- La buena práctica es inicializar los punteros hacia el vacío: el valor especial **nulo** (NULL).

C

```
//Inicializado como puntero nulo:
double *d = NULL;
```

- Aplicaciones de punteros:
  - **Gestión dinámica de memoria:** acceso a la memoria dinámica de heap (limitada por el computador), comparado con el stack (memoria de espacio limitado). Proximamente.
  - **Direccionamiento indirecto:** útil para diseñar estructuras de datos. ELO320 en sí.
  - **Paso por referencia:** pasar elementos por su dirección en memoria para modificarlos donde sea. Se copia dirección y no valor. Próx. Clase!

Punteros en C

- Operador de desreferencia (*): accede al valor en la dirección de memoria **a la que apunta** un puntero.
- Los operadores * y & son opuestos.
- Especificador de punteros: %p.
- *void**: comodín para apuntar a cualquier tipo (coerción, casting).

C

```
int utfsm1 = 1680; //Dos variables de tipo int.
int utfsm2 = 3939;

int *dir = NULL;
dir = &utfsm1; //Puntero a un int, apunta a donde está utfsm1

printf("Dirección dentro de dir: %p\n", dir);
printf("Valor dir: %d\n", *dir);
printf("Dirección de dir: %p\n", &dir);
```

- Nota: los punteros también tienen su propia dirección de memoria (son variables!).

Punteros en C

- El símbolo * no significa lo mismo dependiendo de si es definición, asignación o implementación.

C

```
float b = 1.0;
float * ptr; // * de declaración, solo indicamos que ptr
             // almacena una dirección de memoria de un float

ptr = &b; // sin *, modificamos la direccion a la que apunta ptr.

*ptr = 5.0; // * de desreferencia, se cambia el valor guardado
            // en la dirección de memoria apuntada por ptr

b = *ptr + 1.0; // * de desreferencia, lo que hay guardado donde se apunta + 1.0

float * ptr2 = ptr; // * de declaración.
```

Analogía - Biblioteca

- Preguntamos al bibliotecario o al PC de biblioteca **¿dónde está?** el libro **¿Cómo pasar EDA y MAT023 al mismo tiempo?**.
- Obtenemos la **dirección de la ubicación** del libro.
- Nos dirigimos a esa dirección → **derreferencia**.

Analogía - Biblioteca

- Valor 321 como identificador del libro.
- `int * ubi;` → puntero al libro.
- **ubi** → Pasillo 3, Estante 3, Fila 1.
- ***ubi** → 321.

Analogía - Biblioteca

- Valor 321 como identificador del libro.
- `int * ubi;` → puntero al libro.
- **ubi** → Pasillo 3, Estante 3, Fila 1.
- ***ubi** → 321.

Analogía - Biblioteca

- Valor 321 como identificador del libro.
- `int * ubi;` → puntero al libro.
- **ubi** → Pasillo 3, Estante 3, Fila 1.
- ***ubi** → 321.

Analogía - Biblioteca

- Valor 321 como identificador del libro.
- `int * ubi;` → puntero al libro.
- **ubi** → Pasillo 3, Estante 3, Fila 1.
- ***ubi** → 321.

Analogía - Biblioteca

- Valor 321 como identificador del libro.
- `int * ubi;` → puntero al libro.
- **ubi** → Pasillo 3, Estante 3, Fila 1.
- ***ubi** → 321.

Analogía - Biblioteca

- Valor 321 como identificador del libro.
- `int * ubi;` → puntero al libro.
- **ubi** → Pasillo 3, Estante 3, Fila 1.
- ***ubi** → 321.

Analogía - Biblioteca

- Valor 321 como identificador del libro.
- `int * ubi;` → puntero al libro.
- **ubi** → Pasillo 3, Estante 3, Fila 1.
- ***ubi** → 321.

Analogía - Biblioteca

- Valor 321 como identificador del libro.
- `int * ubi;` → puntero al libro.
- **ubi** → Pasillo 3, Estante 3, Fila 1.
- ***ubi** → 321.

Analogía - Biblioteca

- Valor 321 como identificador del libro.
- `int * ubi;` → puntero al libro.
- **ubi** → Pasillo 3, Estante 3, Fila 1.
- ***ubi** → 321.

Analogía - Biblioteca

- Valor 321 como identificador del libro.
- `int * ubi;` → puntero al libro.
- **ubi** → Pasillo 3, Estante 3, Fila 1.
- ***ubi** → 321.

Aritmética de Punteros

- Indicar el tipo a un puntero sirve para realizar aritmética de punteros.
- Transforma 1 unidad decimal, en 1 unidad segun el tamaño del tipo.
- Podemos saber cuanta memoria hay entre un puntero y el siguiente (si es que existe).
- Ejemplo: Puntero a int avanza de a 4 bytes.

C

```
float num = 2.5;
float *f;
f = &num; //Dirección a num.

printf("Direccion f: %p\n", f); //¿Cuál es la diferencia
printf("Direccion f+1: %p\n", f+1); //entre f y f+1?
```

- Respuesta: La diferencia es de 4 bytes (tamaño de float).

Arreglos y Aritmética de Punteros

- En C un arreglo puede trabajarse con aritmética de punteros.
- Esto se debe a que los arreglos son compatibles con los punteros.
- El compilador hace coerción y esto permite el uso de la aritmética de punteros.

C

```
int a[10], i;
int *pa, *pb;
pa = &a[0]; /* dir. elemento 0 de arreglo a. tamaño 1 int */
pa = a;     /* hace lo mismo que la linea anterior con coerción */
pb = &a;    /* dir. de arreglo a. tamaño 10 int */

for (i=0; i<10; i++){
        printf("%d ", a[i]);
        printf("%d ", *(pa+i));
        printf("%d ", *(a+i));
}
/* los tres for hacen lo mismo */

printf(printf("%d ", *(pb+1)); //¿Qué pasa acá?
```

Funciones y Punteros: Paso por Valor

- Los parametros copian los valores que fueron pasados como parámetros.
- Las variables originales se mantienen inalteradas.

C

```c
#include<stdio.h>

void swap(int a, int b){
        int temp;
        temp = a;
        a = b;
        b = temp;
}

int main(){
        int x=2, y=3;
        swap(x,y);
        printf("x: %d, y: %d\n", x, y); //¿Qué imprime?
        return 0;
}
```

Funciones y Punteros: Paso por Referencia

- Los parametros son punteros locales, pero la derreferencia es la misma local o global.
- Las variables originales se modifican.

C

```c
#include<stdio.h>

void swap(int *a, int *b){ //punteros locales
        int temp;
        temp = *a; //se guarda lo que hay donde apunta a
        *a = *b; //cambio entre los valores donde apuntan los punteros
        *b = temp; //agrego valor guardado, no se cambian los punteros, ojo.
}

int main(){
        int x=2, y=3;
        swap(&x,&y);
        printf("x: %d, y: %d\n", x, y); //¿Qué imprime?
        return 0;
}
```

Funciones y Punteros: Paso por Referencia

- ¿Funciona este tercer caso?

C

```c
#include<stdio.h>

void swap(int *a, int *b){ //punteros locales
        int *temp;
        temp = a;
        a = b;
        b = temp;
}

int main(){
        int x=2, y=3;
        swap(&x,&y);
        printf("x: %d, y: %d\n", x, y); //¿Qué imprime?
        return 0;
}
```

- Respuesta: No, no funciona. Punteros locales.

Funciones y Punteros: Paso por Referencia

- ¿Funciona este cuarto caso?

C

```c
#include<stdio.h>

void swap(int *a, int *b){ //punteros locales
        int *temp;
        temp = a;
        *a = *b;
        *b = *temp;
}

int main(){
        int x=2, y=3;
        swap(&x,&y);
        printf("x: %d, y: %d\n", x, y); //¿Qué imprime?
        return 0;
}
```

- Respuesta: No, se pierde el valor de *a.